

PRACTICEPRO

Codebase Audit & Refactor Roadmap

Applying the Van Cleif Methodology across the full stack

A comprehensive review of security, structure, and polish across the Convex backend, React frontend, and shared infrastructure — with a prioritized, safe-by-default action plan.

Metric	Value	Status
Total Convex functions	423 across 48 modules	MEDIUM
Total React components	339	MEDIUM
Schema tables	65	MEDIUM
TypeScript errors (baseline)	327	HIGH
TypeScript errors (post-fix)	315	SAFE
Build status	Passing (22s)	SAFE
Van Cleif pattern coverage	~35% of modules	HIGH
Critical security findings	6 (3 client-side, 3 server-side)	CRITICAL
Safe fixes applied this session	5 (12 TS errors cleared)	SAFE

Report date: 16 August 2026 | Audited commit: HEAD of origin/main | Auditor: Super Z (independent)

Contents

1. Executive Summary	3
1.1 What the Van Cleif Methodology Is	3
1.2 Headline Findings	4
2. Van Cleif Pattern Coverage Today	4
2.1 Backend Pattern Gap Matrix	5
2.2 Frontend Pattern Gap Matrix	5
3. Backend Audit — Convex	6
3.1 The myFunctions.ts God File	7
3.2 The portals.ts Multi-Tenant Hole	8
3.3 Cross-Module Duplication	8
3.4 Schema Gaps	9
4. Frontend Audit — React + Vite	9
4.1 God Components — Extraction Plan	10
4.2 Security Findings — Stored XSS	10
4.3 Auth & Session UX — Critical Risks	11
4.4 Performance Red Flags	12
5. TypeScript Health	12
5.1 Errors Fixed in This Session	13
5.2 Top 5 Remaining Quick Wins	13
6. Refactor Roadmap — Prioritized & Sequenced	14
6.1 Phase 1 — Security (1-2 days, additive only)	14
6.2 Phase 2 — Data Integrity (3-5 days, additive schema)	15
6.3 Phase 3 — Code Organization (1-2 weeks, behind shims)	15
7. Appendix — Pattern Reference & Recipes	16
7.1 The Idempotency Recipe (from feedback.ts)	16
7.2 The Soft-Delete Recipe (proposed)	17
7.3 The requireFirmUser Recipe (already proven)	18
7.4 Files Modified in This Audit Session	18
7.5 Audit Methodology	19

1. Executive Summary

PracticePro is a mature, multi-product SaaS platform (Vega for legal practice, Atrium for property management, Komplete as the unified tier) built on React 18 + Vite 5 + Convex + Capacitor. The codebase has reached a stage where the polish patterns that were carefully applied to a small number of flagship modules — the Van Cleif methodology — have not been systematically extended to the rest of the system. This audit identifies the gaps, ranks them by impact and safety, and applies a first wave of surgical fixes that are demonstrably safe and worthwhile.

1.1 What the Van Cleif Methodology Is

Named for Van Cleef & Arpels — the French haute joaillerie house famous for the "Sert Mystrieux" invisible setting, where the metalwork is hidden so the gems appear as a continuous surface — the Van Cleif methodology is a set of polish patterns that make a codebase behave predictably under stress. The "invisible setting" metaphor captures the goal: the user never sees the metalwork (the implementation), only the gems (the feature). Practically, the methodology is expressed through seven concrete patterns that are already visible in flagship modules such as `convex/feedback.ts`, `convex/retainerBilling.ts`, `src/constants/identityGuardrails.ts`, and the role-based Terms Acceptance flow.

#	Pattern	What it guarantees	Reference implementation
1	Defense-in-depth gating	Premium feature checks on BOTH client AND server, so a tampered client cannot bypass.	<code>isFirmPremiumRetainerEligible()</code> in <code>retainerBilling.ts:108</code>
2	Idempotency keys	Retried writes return the original result instead of creating duplicates.	<code>idempotencyKey + by_idempotency</code> index in <code>feedback.ts:30</code>
3	Soft-delete with audit trail	Destructive operations become reversible; <code>deletedAt</code> / <code>deletedBy</code> preserved.	<code>deleteFeedbackThread</code> in <code>feedback.ts</code>
4	Identity lock guardrails	AI system prompt + post-processing validator prevent product-identity leakage.	ABSOLUTE IDENTITY LOCK in <code>identityGuardrails.ts</code>
5	Role-based access control	Every mutation verifies caller role via <code>requireFirmUser</code> / <code>requireFounder</code> / <code>requireAdmin</code> .	<code>requireFirmUser</code> in <code>authHelpers.ts</code>
6	Per-product branching	UI text and AI behaviour branch on <code>isProperty</code> for Atrium vs Vega.	<code>useProduct()</code> hook + <code>validateAIResponse(text, isProperty)</code>
7	Defensive auth fallback	Legacy callers without <code>userEmail</code> get an anonymous context (not a hard throw) — fail-soft for backward compatibility.	<code>requireFirmUser</code> at <code>authHelpers.ts:40-56</code>

Table 1.1 — The seven Van Cleif patterns and their canonical implementations.

1.2 Headline Findings

The audit found three categories of work. First, the Convex backend contains two "God files" (myFunctions.ts at 6,279 lines and portals.ts at 5,970 lines) that together account for 151 of the codebase's exports; these files are too large to safely review, modify, or merge against, and they should be split into 30 domain-scoped modules. Second, several Convex modules bypass the Van Cleif auth pattern entirely — most alarmingly, portals.ts has 70+ exports but only ONE calls requireFirmUser, meaning a malicious caller who knows any firmId can post maintenance tickets, send portal messages, modify portal settings, and submit fake payment proofs against any firm. Third, the frontend has accumulated dead code (49 unused shadcn scaffolding files in src/components/ui/ that contribute ~170 of the 327 baseline TypeScript errors) and several untrusted dangerouslySetInnerHTML sites that are not wrapped in sanitize().

Critical takeaway. **CRITICAL** The portals.ts multi-tenant isolation hole is the single highest-priority item in this audit. It is also the safest to fix: adding requireFirmUser calls is additive (only fails closed for missing userEmail, which the legacy fallback already handles) and requires no schema migration. This should be the very next change shipped.

2. Van Cleif Pattern Coverage Today

Before recommending changes, this section maps the existing pattern coverage so the gaps are visible at a glance. The pattern coverage matrix below covers every Convex module with more than 100 lines of code. A green check means the pattern is consistently applied; a warning triangle means partial coverage (e.g. soft-delete exists for one table but not its siblings); a red cross means the pattern is entirely absent.

2.1 Backend Pattern Gap Matrix

Module (LOC)	Idempotency	Soft-delete	Defense-in-depth	RBAC	Verdict
feedback.ts (671) — gold standard	YES	YES	n/a	partial	SAFE
retainerBilling.ts (847)	no	state-machine	YES	YES	SAFE
myFunctions.ts (6,279)	partial	no	partial	YES (48 calls)	HIGH
portals.ts (5,970)	no	partial	no	1 of 70	CRITICAL
founderMetrics.ts (2,394)	no	no	n/a	YES (30 calls)	MEDIUM
broadcasts.ts (565)	no	partial	no	zero	CRITICAL
backups.ts (471)	no	no	no	zero — trigger BackupNow is public	CRITICAL
sentry.ts (1,049) — financial mutations	no	no	no	zero (23 exports)	CRITICAL
legalRepo.ts (283)	no	no	no	zero	HIGH
trustAccount.ts (324)	no	no	partial	YES (4 calls)	HIGH
pushNotifications.ts (434)	no	no	no	zero (claims founder-only in comment)	HIGH
indexer.ts (197)	no	no	no	zero	HIGH
embeddings.ts (105)	no	no	no	zero — clearFarmMemories is public	CRITICAL
ai.ts (93)	no	n/a	no	zero	HIGH
visitorManagement.ts (604)	no	partial (revokedAt)	YES (VMS billing)	partial	MEDIUM
proactive.ts (918)	no	partial (dismissedAt)	n/a	n/a (internal)	SAFE
conversationMemory.ts (334)	no	n/a	n/a	n/a (internal)	SAFE
analytics.ts (277)	no	n/a	n/a	zero (trackEvent public)	MEDIUM

Table 2.1 — Backend pattern coverage. "zero" means the auth helper is never invoked; this is the most dangerous column because unauthenticated mutations are public endpoints on the Convex deployment.

2.2 Frontend Pattern Gap Matrix

The frontend tells a parallel story. Of approximately 176 mutation call sites in src/, only 8 carry an idempotencyKey (concentrated in MessagesView and FeedbackForm). The remaining 168 sites are at double-submit risk — particularly on mobile networks where Capacitor retries are common. The table below highlights the highest-leverage gaps.

Component (mutation calls)	Idempotency	isProperty branch	useFeatures gate	Submit guard	Priority
AloaChat.tsx (3 calls — saveMessage, createConv, deleteConv)	no	YES	no	partial	CRITICAL
DraftProEditor.tsx (AI drafting, save-as-template)	no	YES	no	partial	HIGH
TenantPortal.tsx (4 calls)	no	implicit	YES	YES	HIGH
MessagesView.tsx (4 calls — admin reply paths missing)	partial	YES	no	partial	HIGH
TermsAcceptance.tsx (recordTermsAcceptance)	no	role-based	n/a	FIXED this session	SAFE
VisitorPortal.tsx (generateVisitorToken, revoke)	no	n/a	no	no	HIGH
PaymentGatewayModal.tsx (createSubscriptionRequest)	no	n/a	no	partial	CRITICAL
ModalManager.tsx (recordConsent, addLedgerEntry)	no	n/a	no	partial	HIGH
BillingMonitorView.tsx (5 financial writes)	no	n/a	partial	partial	CRITICAL
TrustAccountTab.tsx (recordTransaction, deleteTransaction)	no	n/a	no	partial	HIGH
DataProvider.tsx (4 central mutations)	no	n/a	n/a	n/a	CRITICAL
AuthContext.tsx (5 mutations incl. deleteAccount)	no	n/a	n/a	no inflight state	CRITICAL

Table 2.2 — Frontend pattern coverage for the 12 highest-leverage components. The DataProvider row is significant because it is the central choke point: adding idempotency here propagates to every consumer automatically.

3. Backend Audit — Convex

The Convex backend has 48 TypeScript modules totalling approximately 27,000 lines. Two files account for over 12,000 of those lines: myFunctions.ts (6,279 LOC, 81 exports) and portals.ts (5,970 LOC, 70 exports). Both files mix unrelated domains — auth, billing, messaging, maintenance tickets, invites, broadcasts, analytics — into single mega-files that are too large to safely review. This section proposes a domain-driven split that respects the existing module boundaries and minimizes import churn.

3.1 The myFunctions.ts God File

A line-count breakdown reveals 18 distinct domains crammed into one file. The largest cluster is Auth (26 exports, ~1,300 LOC) followed by Generic CRUD (7 exports), Subscriptions (9 exports), and Email Service (7 exports). The proposed split moves each cluster into a dedicated module under convex/, leaving myFunctions.ts as a thin re-export shim for backward compatibility during the migration. The split targets below are sized so that no new module exceeds ~700 lines.

Domain cluster	Exports	Representative exports	Proposed module
Auth & users	26	verifyLogin, startSignup, createUser, deactivateTeamMember, validateInviteCode, joinFirm	convex/auth.ts
Firm CRUD	8	getFirmData, createFirm, updateFirmSettings, deleteFirm, getJoinedFirms	convex/firms.ts
Generic CRUD	7	createItem, updateItem, deleteItem, forceDeleteItem, purgeFirmData	convex/genericCrud.ts
Chat / messaging	8	sendChatMessage (has idempotency), saveAloaMessage, getAloaConversations	convex/chat.ts
Email service (Brevo)	7	sendVerificationEmail, sendRecoveryEmail, sendBreachNotification	convex/emailService.ts
Subscriptions	9	createSubscriptionRequest, approveSubscriptionRequest, expireTrials	convex/subscriptions.ts
Add-ons / VMS billing	9	startVmsAddonTrial, activateVmsAddon, cancelVmsAddon	convex/addons.ts
Admin user ops	3	adminSearchUsersByEmail, adminDeleteUser, adminForceVerify	convex/adminUsers.ts
Cascade deletes	3	deleteMatterCascade, deletePropertyCascade, deleteContactCascade	convex/cascadeDeletes.ts
Broadcast engine	4	getAllUsersForBroadcast, createBroadcastNotification	merge into convex/broadcasts.ts
Presence	4	sendHeartbeat, getActivePeers, diagnoseConnectivity	convex/presence.ts
Tasks	3	createTask, updateTask, updateTaskStatus	convex/tasks.ts
Matters / properties reads	3	getMatterDetails, getPropertyDetails, getPropertyById	convex/matters.ts + convex/properties.ts
Other (terms, notifications, WhatsApp quota, etc.)	12	recordTermsAcceptance, markNotificationsAsRead, incrementWhatsAppQuota	5 small modules

Table 3.1 — Domain-cluster split for myFunctions.ts. 81 exports total; largest cluster (Auth) is 32% of the file. Migration can be done cluster-by-cluster behind a re-export shim.

3.2 The portals.ts Multi-Tenant Hole

Severity: **CRITICAL** Effort to fix: low Risk to fix: very low

A grep for `requireFirmUser|requireFounder|requireAdmin` in `portals.ts` returns only two matches: the import statement at line 4 and one call at line 307 inside `completePortalTask`. The remaining 69 exports accept a client-supplied `firmId` argument and write directly to the database. A malicious caller who knows any `firmId` (which is exposed in portal invite URLs, in user JWTs, and in many public query responses) can: post maintenance tickets against any firm (`createMaintenanceTicket` L19); send portal messages impersonating any user (`sendPortalMessage` L3941); modify portal settings for any firm (`updateFirmPortalSettings` L5003); and submit fake payment proofs (`submitPaymentProof` L4644). The fix is purely additive: prepend each mutation with `const ctx = await requireFirmUser(args.userEmail, args.firmId);` — the existing defensive fallback in `authHelpers.ts:40-56` already handles legacy callers that do not pass `userEmail`.

3.3 Cross-Module Duplication

Several utility functions are duplicated across modules with divergent behaviour. The most dangerous case is `computeNextBillingDate`, which exists in three copies: the canonical implementation in `retainerBilling.ts:62` (correctly clamps to month-end), and two private re-implementations in `myFunctions.ts:5656` and `founderMetrics.ts:1784` that silently produce wrong results for end-of-month dates. A consolidation pass should extract these into a `convex/dateUtils.ts` module and a `convex/moneyUtils.ts` module (the latter already exists with `roundMoney()` but is only used in two places — the other 14 inline `Math.round(num * 100) / 100` sites should be replaced with calls to it).

Duplicated helper	Copies	Files (line)	Consolidation target
<code>computeNextBillingDate</code> (calendar math)	3 (divergent!)	<code>retainerBilling.ts:62</code> , <code>myFunctions.ts:5656</code> , <code>founderMetrics.ts:1784</code>	<code>convex/dateUtils.ts</code>
<code>Math.round(num*100)/100</code> (money rounding)	3	<code>moneyUtils.ts:19</code> , <code>myFunctions.ts:3397</code> , <code>founderMetrics.ts:2319</code>	use <code>moneyUtils.roundMoney</code>
<code>new Date().toISOString().split("T")[0]</code>	14 occurrences	<code>analytics</code> , <code>founderMetrics</code> , <code>visitorManagement</code> , <code>crons</code> , <code>proactive</code> , <code>backups</code>	<code>dateUtils.isoDate()</code>
<code>generatePortalAccessToken</code> (UUID v4)	1 file, 5 calls	<code>portals.ts:11</code>	extend <code>convex/portalSecurity.ts</code>
<code>BREVO_SENDER</code> + HTML email wrapper	3 sets	<code>myFunctions.ts:3707</code> , <code>portals.ts:5631</code> , <code>portals.ts:5911</code>	<code>convex/emailService.ts</code>
<code>logSecurityEventInline</code> (try/catch security sink)	3 sites	<code>securityHelpers.ts:98</code> , <code>authHelpers.ts:60/101/120</code>	export from <code>securityHelpers</code>
Inline notification-insert block (8 lines)	13+ sites	<code>myFunctions.ts</code> (8), <code>portals.ts</code> (5+), <code>feedback.ts</code> (1)	<code>notifyUser()</code> helper

Table 3.2 — Cross-module duplication catalog. The "divergent!" annotation on `computeNextBillingDate` is the most concerning: the three copies produce different results for the same input, which means billing dates are silently wrong for any code path that uses the non-canonical copies.

3.4 Schema Gaps

The archive table (schema.ts:672) was created as the soft-delete infrastructure — it is designed to snapshot a row before destructive deletion. Today, no mutation populates it. Meanwhile, 18 tables are hard-deleted via `db.delete()` calls scattered across `myFunctions.ts` (18 sites), `portals.ts` (5 sites), and individual modules (`sentry.ts`, `indexer.ts`, `embeddings.ts`, `securityHelpers.ts`). The most impactful tables to convert are: `matters`, `contacts`, `properties` (all deleted via cascade functions that destroy legal/financial records); `invoices` (NDPA financial-audit requirement); `trust_transactions` (hard-deleted by `deleteTrustTransaction` in `trustAccount.ts:303` — destroys the financial audit trail); `users` (should set `deactivatedAt`, which already exists, instead of hard delete); and `chatMessages` (has `isDeleted` flag but no `deletedAt/deletedBy` audit fields). Conversion is additive: add `deletedAt`, `deletedBy`, and `deletedReason` fields with a `by_firm_deleted` index, then change `db.delete()` calls to patch operations.

4. Frontend Audit — React + Vite

The frontend consists of 339 React components totalling roughly 151,000 lines of TypeScript/TSX. Nine components exceed 1,500 lines and account for ~25,000 lines on their own. The largest is `TenantPortal.tsx` at 3,879 lines, which packs 10 distinct tab views, 13 inline SVG icon components, Paystack checkout setup, document printing, and feedback submission into a single function component. This is the canonical "god component" anti-pattern: it cannot be safely reviewed, it re-renders on every keystroke (zero `useMemo`, zero `useCallback`), and any change risks breaking unrelated tabs.

4.1 God Components — Extraction Plan

File (LOC)	Inline sub-components to extract	Target directory	Root LOC after
TenantPortal.tsx (3,879)	DashboardTab, NoticesTab, LedgerTab, ReceiptsTab, MaintenanceTab, MessagesTab, PaymentsTab, DocumentsTab, HelpAndSupportTab + 13 inline SVG icons	src/components/tenant/tabs/* .tsx	~250
DraftProEditor.tsx (3,798)	28 inline SVG icons (1,100 LOC), FONTS/SIZES constants, ToolbarGroup/ToolbarBtn, AutoPagination plugin, CitationCard, FillPlaceholdersModal, RedraftModal, SaveAsTemplateModal, useDraftStream hook	src/components/documents/typetap/* + hooks/*	~600
AloaChat.tsx (3,455)	useLiveAudio hook, useChatScrollAnchor hook, ToolDispatcher, MessageList, Composer, InsightsPanel + ZERO useMemo today	src/components/aloa/* + hooks/*	~400
MessagesView.tsx (3,315)	ChatWindow, MessageContent, InlineTicketReply, SectionHeader, 4 inbox sub-sections	src/components/messaging/*	~500
PropertyDetailView.tsx (2,834)	UnitSwitcher, PropertyTicketsTab, PropertyNoticesTab, LinkedMattersTab, lease calculators	src/components/details/property/*	~600
ClientDashboard.tsx (2,020)	StageBadge, SummaryCard, TabButton, EmptyState, FileTypeIcon + 6 tab bodies	src/components/client/tabs/*	~400
PropertyForm.tsx (2,012)	useUnitRentalCalc hook, AccordionSection (reusable), 7 section sub-forms	src/components/forms/property/sections/*	~500
LandingPage.tsx (1,581)	NavBar, Footer, HubHero, HomeSection, FeaturesSection, PricingSection, SceCalculatorModal + 3 button primitives	src/components/marketing/*	~100
App.tsx (1,531)	ViewWrapper, MainContent, AppRouter, useAuthRoutes / usePortalRouteParser / useIdleAutoLogout hooks + lazyWithReload util	src/components/layout/* + utils/*	~400

Table 4.1 — Extraction plan for the 9 god components. The "Root LOC after" column assumes all inline sub-components are extracted; total reduction is ~22,000 lines moved out of monolithic files into focused, testable modules.

4.2 Security Findings — Stored XSS

Ten files use dangerouslySetInnerHTML. Six of them route through parseAloaMarkdown which calls sanitize() (the safe path). Four of them inject untrusted HTML directly without sanitization, creating stored-XSS or prompt-injection attack vectors. The fix is mechanical: wrap each untrusted input in sanitize() from src/utils/sanitization.ts (which uses DOMPurify with a strict allowlist and is already used by AloaChat, NoteDetails, and ResourcesPage).

File:Line	Source of HTML	Risk	Status
DocumentDetailView.tsx:252	Server-stored document content	Stored XSS — user A saves doc with <script>, user B views it	SAFE FIXED
HtmlPagePreview.tsx:173	DraftPro editor output	Same as above (DraftPro content)	SAFE FIXED

File:Line	Source of HTML	Risk	Status
HtmlPagePreview.tsx:8 15	Paginated version of DraftPro output	Same as above	SAFE FIXED
ResearchStudio.tsx:145	AI research output	Prompt-injection XSS — LLM can be tricked into emitting 	SAFE FIXED
ui/chart.tsx:83	Recharts internal CSS	Low (trusted library)	LOW no action
AloaChat.tsx:2877	AI markdown via parseAloaMarkdown	Safe (routes through sanitize)	SAFE

Table 4.2 — Untrusted HTML injection sites. All four critical sites were fixed in this audit session by wrapping the HTML in `sanitize()` before injection.

4.3 Auth & Session UX — Critical Risks

The authentication layer has three architectural risks that pre-date this audit and require a coordinated fix. They are flagged here as recommendations rather than applied because the fix touches every portal route, every Convex caller, and four `localStorage` keys — too large a blast radius for an audit-session patch. A dedicated security sprint should be scheduled to address them.

Risk	Detail	Fix scope
CRITICAL Email-as-token	<code>AuthContext.getInitialToken()</code> returns the user's lowercased email as the session token. No JWT, no opaque token, no expiry. Stored in 4 <code>localStorage/sessionStorage</code> slots in plaintext. Anyone who exfiltrates an email can impersonate.	Issue opaque server-side session token; rotate on each login; expire after 30d.
CRITICAL Unsigned <code>?impersonate= param</code>	Appending <code>?impersonate=admin@firm.com</code> to any URL writes that email to <code>localStorage</code> and returns it as the token. No signature, no nonce, no founder-secret. Universally accessible.	Require a signed, short-lived <code>impersonateToken</code> issued by a founder-only mutation.
HIGH Plaintext PII in <code>localStorage</code>	<code>practicepro_cached_user</code> stores email, <code>firmId</code> , <code>barNumber</code> , <code>accessibleMatterIds</code> , <code>portalAccessToken</code> in plaintext. Violates NDPA 25 (data minimization). Any XSS payload can read it.	Encrypt with WebCrypto + a per-session key derived from a server-issued nonce.
HIGH Gemini API key in plaintext	<code>practicepro_custom_gemini_key</code> is synced from server to <code>localStorage</code> without encryption. Any XSS can exfiltrate it (and bill the firm).	Keep key server-side only; proxy all Gemini calls through a Convex action.
MEDIUM <code>useFeatures</code> permissive default	When <code>firmDetails.subscriptionPlan</code> is undefined (still loading), portal users default to Complete (full access). A portal user whose firm data fails to load sees ALL premium features unlocked until the firm data arrives.	Default to Core (deny-by-default); unlock on firm-data arrival.

Table 4.3 — Auth & session UX risks ranked by severity. The top two rows are pre-existing critical risks that require a dedicated security sprint; they are too large a blast radius for an audit-session patch.

4.4 Performance Red Flags

Two performance issues stand out. First, AloaChat.tsx — 3,455 lines — has zero useMemo and only two useCallback declarations. Every keystroke in the composer re-renders the entire chat history, recreates the Web Audio graph, and rebuilds the citation registry. This is the single highest-impact memoization fix in the codebase. Second, only 5 of approximately 25 top-level views are lazy-loaded (App.tsx:43-62 — the lazyWithReload wrapper is excellent). The other 20 views — including the 3,879-line TenantPortal, the 2,020-line ClientDashboard, and the 1,581-line LandingPage — are statically imported, shipping ~12,000 lines of code in the main chunk that is never executed for users who do not visit those routes.

Issue	Impact	Fix
AloaChat zero useMemo/useCallback	Every keystroke re-renders entire chat history (3,455 LOC). Audio graph + citation registry recreated per render.	Wrap messages, citationRegistryRef, parseAloaMarkdown output in useMemo. Risk: stale-closure bugs if deps wrong — needs careful testing.
20 views statically imported	~12,000 LOC shipped in main chunk that is never executed for most users.	Wrap TenantPortal, ClientDashboard, PropertyDetailView, SettingsView, LandingPage in lazyWithReload + Suspense. Pattern already established.
Duplicate notification queries	Header.tsx and NotificationCenter.tsx both fire getUserNotifications + getUnreadNotificationCount on every authenticated render.	Lift to a useNotifications() hook; single source of truth.
BillingMonitorView over-fetching	Fires getOutboxForFirm + getOutboxStats for ALL firms including Atrium (which has no retainer billing).	Skip query when !isProperty or !canUseRetainerAutoBilling.
useScrollParallax no rAF throttle	LandingPage scroll listener fires on every pixel; no requestAnimationFrame throttle.	Wrap in rAF; reuse FlashlightBackground pattern from ConvexErrorBoundary.tsx:78-92.

Table 4.4 — Top 5 performance red flags. The AloaChat memoization is the highest-leverage fix but also the riskiest — incorrect dependency arrays will cause stale-closure bugs in the streaming audio and citation paths.

5. TypeScript Health

The codebase has a baseline of 327 TypeScript errors. This audit applied five surgical fixes that reduced the count to 315 (12 errors cleared) without any behavioural change. The remaining errors are concentrated in three areas: 49 dead shadcn scaffolding files in src/components/ui/ that reference packages not installed in this project (radix-ui, clsx, tailwind-merge, class-variance-authority); the gemini service model union that was missing the "research" literal; and a handful of nullability issues in utility files. None of the remaining errors are blockers — the production build passes cleanly via Vite — but clearing them would significantly improve developer experience and the reliability of IDE refactors.

5.1 Errors Fixed in This Session

Fix	File	Errors cleared	Risk
Removed duplicate useQuery + api imports (wrong path was ../convex/_generated/api instead of ../../convex/_generated/api)	src/components/Header.tsx:5-6	5	SAFE
Added "research" literal to modelPreference union (already used in code at lines 321, 364, 608, 622)	src/services/geminiService.ts:286, 575	5 (cascade)	SAFE
Fixed cachedIsNative nullability — module-level let cannot be narrowed by TS after assignment	src/utls/capacitor.ts:30-32, 44-45	1	SAFE
Wrapped 4 untrusted dangerouslySetInnerHTML sites in sanitize()	DocumentDetailView:252, HtmlPagePreview:173/815, ResearchStudio:145	0 (security fix)	SAFE
Added isSubmitting guard to TermsAcceptance.handleAccept + disabled Accept button during await	src/components/TermsAcceptance.tsx:148-182, 237-243	0 (UX fix)	SAFE

Table 5.1 — Five surgical fixes applied in this audit session. All are additive (no behavioural change for legitimate callers) and verified by a clean Vite production build (22s). The build was confirmed to pass before and after the changes.

5.2 Top 5 Remaining Quick Wins

The remaining 315 errors break down into clear categories. The single biggest win — accounting for approximately 170 of the 315 errors — is to delete the 49 dead shadcn scaffolding files in `src/components/ui/`. These files reference packages that are not installed in this project (verified: `radix-ui`, `clsx`, `tailwind-merge`, `class-variance-authority` are all absent from `package.json`). They were scaffolded by `shadcn-ui` at some point but never wired in. The only file in that directory that is actually imported by the rest of the codebase is `ConfirmDialog.tsx` (15 importers verified). The other 49 files plus `CognitiveGuidance.tsx` (zero importers) can be deleted in a single commit.

#	Fix	Errors cleared	Effort	Risk
1	Delete 49 dead shadcn files + <code>CognitiveGuidance.tsx</code> (zero importers verified)	~170 (55% of remaining)	1 hour	SAFE
2	Add <code>idempotencyKey</code> + attachments to <code>sendMessage</code> arg type (client passes both; server type only knows <code>idempotencyKey</code>)	8 (3 direct + 5 cascade)	15 min	SAFE
3	Add <code>enabled</code> field to <code>PostHog</code> config type in <code>src/utls/analytics.ts:35</code>	1	5 min	SAFE
4	Fix <code>LocalNotificationSchema.data</code> access in <code>src/utls/notifications.ts:137</code>	1	10 min	SAFE

#	Fix	Errors cleared	Effort	Risk
5	Fix string null to string undefined in propertyPayload.ts:106-108	3	10 min	SAFE

Table 5.2 — Top 5 remaining quick wins. Together they would clear ~183 of the 315 remaining errors (58%) in under 2 hours, all with very low risk.

6. Refactor Roadmap — Prioritized & Sequenced

The recommendations are grouped into three phases, ordered by safety and dependency. Phase 1 is purely additive security work — no schema changes, no behavioural change for legitimate callers — and should ship first. Phase 2 adds data-integrity infrastructure (idempotency keys, soft-delete fields) that requires additive schema migrations but no data backfill. Phase 3 is pure code organization (module splits, god component extraction, dead code removal) and can be done incrementally behind re-export shims. Each phase is designed so that any individual item can be shipped independently of the others.

6.1 Phase 1 — Security (1-2 days, additive only)

Goal: Close the multi-tenant isolation hole in portals.ts and add auth gates to the other unauthenticated modules. Constraint: All changes are additive — they only fail closed for missing userEmail, which the legacy fallback already handles. No schema migration, no behavioural change for legitimate callers.

#	Refactor	Impact	Safety	Effort
P1.1	Add requireFirmUser to 40 unauthenticated portals.ts mutations	CRITICAL multi-tenant isolation	very high	4 hours
P1.2	Add requireFirmUser/requireFounder to sentry.ts (23 financial mutations)	CRITICAL financial integrity	high	1 day
P1.3	Add requireFounder to embeddings.clearFirmMemories, indexer.deleteAloaDocument/saveAloaDocument	CRITICAL firm-data destruction	very high	2 hours
P1.4	Add requireFounder to pushNotifications.notifyAppUpdate/sendToUsers/sendTestPush/sendTestPushToUser	HIGH push spam vector	very high	2 hours
P1.5	Add requireFounder to salesInquiries.listSalesInquiries/updateInquiryStatus	MEDIUM founder-data leak	very high	1 hour
P1.6	Add requireFounder to backups.triggerBackupNow	CRITICAL unauthenticated backup trigger	very high	30 min
P1.7	Add requireFounder to analytics.getDashboardData/getFirmsList	MEDIUM founder-data leak	very high	1 hour

Table 6.1 — Phase 1 priorities. All additive, all close fail-closed for missing userEmail. Estimated total: 1-2 days; should ship as a single PR.

6.2 Phase 2 — Data Integrity (3-5 days, additive schema)

Goal: Add idempotency keys to all retryable writes; convert hard-deletes to soft-deletes with audit trail. Constraint: Schema changes are additive (new optional fields + new indexes); existing rows default to null and behave as before. No data backfill required. Pattern is already proven in `feedback.ts` and `sendMessage` — this phase simply extends it to the rest of the codebase.

#	Refactor	Impact	Safety	Effort
P2.1	Add <code>idempotencyKey</code> + <code>by_idempotency</code> index to 5 critical tables (<code>payment_proofs</code> , <code>subscriptionRequests</code> , <code>subscriptionAddons</code> , <code>termsAcceptance</code> , <code>tasks</code>)	HIGH double-submit on financial/legal	high	1 day
P2.2	Convert <code>deleteItem</code> + <code>forceDeleteItem</code> + 3 cascade deletes to soft-delete via archive table (already exists at <code>schema.ts:672</code>)	HIGH NDPA compliance + reversibility	medium (large blast radius — do table-by-table)	2 days
P2.3	Convert <code>trustAccount.deleteTrustTransaction</code> to soft-delete	HIGH financial audit trail	high (single mutation)	2 hours
P2.4	Add <code>deletedAt/deletedBy</code> to <code>chatMessages</code> (already has <code>isDeleted</code> flag) and to <code>aloeConversations/aloeMessages</code>	MEDIUM conversation audit trail	high	4 hours
P2.5	Convert <code>adminDeleteUser</code> + <code>deleteAccount</code> to set <code>deactivatedAt</code> (already exists) instead of <code>db.delete()</code>	HIGH user audit trail + GDPR/NDPA right-to-be-forgotten	high	3 hours

Table 6.2 — Phase 2 priorities. The archive table at `schema.ts:672` is already provisioned but unused today — this phase lights it up. P2.2 has the largest blast radius (touches ~18 deletion sites across `myFunctions.ts` + `portals.ts`) and should be done table-by-table to limit regression risk.

6.3 Phase 3 — Code Organization (1-2 weeks, behind shims)

Goal: Split the two God files; extract god components; remove dead code; consolidate duplicated helpers. Constraint: All changes are pure code reorganization — no behavioural change. Module splits should ship behind a re-export shim (`myFunctions.ts` becomes a thin file that re-exports from the new domain modules) so callers do not need to update imports in lockstep. God component extraction can be done one component at a time. Dead code removal is the lowest-risk item and can ship first to clear ~170 TypeScript errors as a quick morale win.

#	Refactor	Impact	Safety	Effort
P3.1	Delete 49 dead shadcn files + <code>CognitiveGuidance.tsx</code> (zero importers verified)	MEDIUM ~170 TS errors + bundle shrinks ~50KB	very high	1 hour
P3.2	Lazy-load <code>TenantPortal</code> , <code>ClientDashboard</code> , <code>PropertyDetailView</code> , <code>SettingsView</code> , <code>LandingPage</code> via <code>lazyWithReload</code>	HIGH ~12,000 LOC out of main chunk	high	2 hours
P3.3	Consolidate <code>formatNaira</code> + <code>formatDate</code> into <code>src/utls/formatting.ts</code> (13 inline copies)	MEDIUM consistency + bug-fix propagation	very high (pure mechanical)	1 hour

#	Refactor	Impact	Safety	Effort
P3.4	Extract DraftProEditor inline SVGs (28 icons, ~1,100 LOC) to lucide-react or toolkit/Icon	MEDIUM file drops 3,798 to ~2,700 LOC	high	2 hours
P3.5	Split myFunctions.ts into 18 domain modules behind re-export shim	MEDIUM maintainability + build time	medium (80 import sites to update)	3 days
P3.6	Split portals.ts into 12 portal/* modules behind re-export shim	MEDIUM maintainability	medium (50+ import sites)	2 days
P3.7	Extract TenantPortal 10 tab sub-components	MEDIUM file drops 3,879 to ~250 LOC	medium (firm-scoped queries need careful prop passing)	1 day
P3.8	Add useMemo to AloaChat message list + citation registry (ZERO useMemo today)	HIGH single biggest perf win	medium (stale-closure risk in streaming paths)	2 hours + testing
P3.9	Consolidate computeNextBillingDate (3 divergent copies) into convex/dateUtils.ts	HIGH silent billing-date bugs today	very high (pure functions)	2 hours
P3.10	Hoist useNotifications hook (eliminates duplicate notification queries)	MEDIUM network efficiency	high	3 hours

Table 6.3 — Phase 3 priorities. P3.1 ships first as a quick morale win (170 TS errors gone in 1 hour). P3.5 and P3.6 are the largest items and should be done incrementally (one cluster at a time) behind re-export shims.

7. Appendix — Pattern Reference & Recipes

7.1 The Idempotency Recipe (from feedback.ts)

The idempotency pattern is already battle-tested in this codebase. The recipe has three steps: add an optional idempotencyKey arg and a by_idempotency index to the schema; at the start of the mutation, query the index and return the existing record if a match is found; otherwise insert with the key. The code snippet below shows the canonical implementation from convex/feedback.ts that should be replicated for the 5 critical tables identified in Phase 2.

```
// 1. Schema (convex/schema.ts)
user_feedback: defineTable({
  // ... existing fields ...
  idempotencyKey: v.optional(v.string()),
}).index("by_idempotency", ["idempotencyKey", "_creationTime"]),

// 2. Mutation (convex/feedback.ts)
export const submitFeedback = mutation({
  args: {
    // ... existing args ...
    idempotencyKey: v.optional(v.string()),
  },
  handler: async (ctx, args) => {
    // Dedup check — return existing record if key matches
    if (args.idempotencyKey) {
      const existing = await ctx.db
```

```

        .query("user_feedback")
        .withIndex("by_idempotency", q =>
            q.eq("idempotencyKey", args.idempotencyKey))
        .first();
        if (existing) return existing._id;
    }
    // ... normal insert path ...
},
});

// 3. Client (src/components/forms/FeedbackForm.tsx)
import { v4 as uuidv4 } from 'uuid';
const idempotencyKey = useMemo(() => uuidv4(), []);
await submitFeedback({ /* ...args... */ idempotencyKey });

```

7.2 The Soft-Delete Recipe (proposed)

The soft-delete pattern extends the existing archive table infrastructure. Instead of calling `db.delete()`, the mutation should: snapshot the row to archive, patch the original row with `deletedAt/deletedBy/deletedReason`, and update all list queries to filter out rows where `deletedAt` is truthy. A helper module (`convex/softDelete.ts`) should encapsulate this so the pattern is consistent across modules.

```

// 1. Schema additions (per table)
matters: defineTable({
    // ... existing fields ...
    deletedAt: v.optional(v.number()),
    deletedBy: v.optional(v.string()),
    deletedReason: v.optional(v.string()),
}).index("by_firm_deleted", ["firmId", "deletedAt"]),

// 2. Proposed helper (convex/softDelete.ts)
export async function softDelete(
    ctx: QueryCtx,
    table: string,
    id: Id<string>,
    deletedBy: string,
    reason?: string,
) {
    const row = await ctx.db.get(id);
    if (!row || row.deletedAt) return; // idempotent
    await ctx.db.insert("archive", {
        originalTable: table,
        originalId: id,
        snapshot: JSON.stringify(row),
        deletedAt: Date.now(),
        deletedBy,
        reason,
    });
    await ctx.db.patch(id, {
        deletedAt: Date.now(),
        deletedBy,
        deletedReason: reason,
    });
}

// 3. List queries filter out soft-deleted rows
const activeMatters = await ctx.db
    .query("matters")
    .withIndex("by_firm_deleted", q =>

```

```
q.eq("firmId", firmId).eq("deletedAt", undefined))
.collect();
```

7.3 The requireFirmUser Recipe (already proven)

The auth gate pattern is already implemented in `convex/authHelpers.ts`. The defensive fallback at line 40-56 returns an anonymous context (`firmId: "", user: null`) when `userEmail` is missing — this is the legacy-compatibility path that makes the fix safe to add to existing mutations without breaking callers that have not yet been updated to pass `userEmail`. To close a portal mutation, prepend the body with a single line that destructures the auth context.

```
// Before (portals.ts:19 – multi-tenant hole)
export const createMaintenanceTicket = mutation({
  args: {
    firmId: v.string(),    // client-supplied – UNVERIFIED
    // ... other args ...
  },
  handler: async (ctx, args) => {
    // directly writes – no auth check
    return await ctx.db.insert("maintenance_tickets", { ... });
  },
});

// After (one-line fix)
export const createMaintenanceTicket = mutation({
  args: {
    firmId: v.string(),
    userEmail: v.optional(v.string()), // new
    // ... other args ...
  },
  handler: async (ctx, args) => {
    const { firmId } = await requireFirmUser(ctx, args.userEmail, args.firmId);
    // ^ throws if userEmail belongs to a different firm
    return await ctx.db.insert("maintenance_tickets", { firmId, ... });
  },
});
```

7.4 Files Modified in This Audit Session

The five surgical fixes applied during this audit session are listed below. All changes were verified by a clean Vite production build (22s) and a TypeScript error reduction from 327 to 315. None of the changes alter runtime behaviour for legitimate callers; they only add safety (XSS sanitization, submit guards) and fix pre-existing type-system drift (duplicate imports, missing union literals, nullability).

File	Change	Verified by
<code>src/components/Header.tsx</code>	Removed duplicate <code>useQuery</code> + <code>api</code> imports (wrong path)	<code>tsc --noEmit</code> + vite build
<code>src/services/geminiService.ts</code>	Added "research" literal to <code>modelPreference</code> union (2 sites)	<code>tsc --noEmit</code>
<code>src/utils/capacitor.ts</code>	Fixed <code>cachedIsNative</code> nullability (use local var)	<code>tsc --noEmit</code>
<code>src/components/details/DocumentDetailView.tsx</code>	Wrapped <code>dangerouslySetInnerHTML</code> in <code>sanitize()</code> + import	vite build

File	Change	Verified by
src/components/documents/HtmlPagePreview.tsx	Wrapped 2 dangerouslySetInnerHTML sites in sanitize()	vite build
src/components/research/ResearchStudio.tsx	Wrapped dangerouslySetInnerHTML in sanitize() + import	vite build
src/components/TermsAcceptance.tsx	Added isSubmitting guard + disabled Accept button during await	vite build

Table 7.1 — Files modified in this audit session. All changes are surgical and verified.

7.5 Audit Methodology

This audit was conducted by two parallel exploration agents (one for the Convex backend, one for the React frontend) using read-only file inspection. Each agent produced a structured report with line-numbered citations; the findings were cross-referenced against the existing worklog.md entries to verify which patterns were intentionally applied in previous sessions. The five safe fixes were applied manually by the main agent and verified by the production build. No files were deleted, no schema was modified, and no Convex mutations were altered. The full audit transcript is preserved in the conversation history for future reference.

The methodology prioritized safety above all else. Every recommendation is tagged with an explicit safety rating (very high / high / medium) and an effort estimate. The phase ordering ensures that no item in a later phase depends on an item in an earlier phase being shipped first — each item can be deployed independently. The user should review the roadmap with their engineering team and select items for implementation based on their own risk tolerance and roadmap priorities.